Failure-Risk Prediction for a Metro Train Compressor Using Machine Learning

1. Abstract

This project applies supervised machine learning to the MetroPT-3 air compressor dataset for predictive maintenance of a metro train compressor. The task is binary classification of 1-minute sensor windows: `0` means normal operation and `1` means failure-risk/anomaly. A window is labeled failure-risk if its timestamp occurs from 1 hour before a documented failure start time through the documented failure end time.

The main evaluation uses an event-aware blocked split. Failure events 1 and 2 are used for training, failure event 3 is used for validation and threshold selection, and failure event 4 is held out for final testing. This split is essential because the processed data contains 5,200 positive 1-minute windows, but those windows come from only 4 independent documented failure events. They are not 5,200 independent compressor failures.

Five supervised models were compared: Logistic Regression baseline, Decision Tree, Random Forest, Extra Trees, and HistGradientBoostingClassifier. The selected final model was the Logistic Regression baseline with validation-selected threshold 0.61. On held-out event 4, the model achieved high accuracy only because the test block was dominated by normal windows. It failed to detect the failure-risk class: precision, recall, F1-score, and F2-score were all 0.0, with 330 false negatives and 0 true positives. The main conclusion is that realistic event-aware testing exposed failed generalization to held-out failure event 4. The current model is not deployment-ready.

2. Introduction and Motivation

Railway compressor failures can create repair cost, service disruption, and operational risk. A useful predictive-maintenance model should therefore prioritize identifying failure-risk windows, even if doing so creates some extra inspection burden. In this setting, a false negative is more serious than a false positive because a missed failure-risk window can mean a missed opportunity to inspect or repair the compressor before a failure affects service.

The machine learning task is supervised binary classification:

- `0` = normal operation
- `1` = failure-risk/anomaly

This project focuses on a reproducible course workflow rather than a production deployment. The report emphasizes realistic event-aware testing, honest discussion of non-generalization, and clear separation between validation and test data. Project decisions, reproducibility notes, commands, problems, fixes, and verified results are tracked in `PROJECT_LOG.md`. Additional report-ready explanations are collected in `report/REPORT_NOTES.md`.

3. Related Work or Background

Predictive maintenance uses sensor data to detect abnormal equipment behavior before or during a failure

period. For compressor systems, pressure, temperature, current, and discrete operating-state measurements can reflect changes in system behavior. Machine learning models can learn associations between engineered sensor features and labeled failure-risk periods.

Time-series predictive maintenance requires careful validation. Randomly splitting individual windows can leak information because nearby windows from the same operating period or the same failure event may appear in both training and testing. This can make performance look stronger than it would be on a future independent failure event. For that reason, this project treats the event-aware blocked split as the main realistic evaluation and treats the stratified window-level split only as an optimistic baseline.

4. Dataset Description

The dataset is the MetroPT-3 Dataset from the UCI Machine Learning Repository. It contains timestamped sensor readings from an air compressor system, including pressure, temperature, motor current, and other operational measurements.

The confirmed raw file used locally is `data/raw/MetroPT3(AirCompressor).csv`. After running `python -m src.preprocessing`, the raw data was confirmed to contain 1,516,948 rows and 17 columns. The timestamp column is named `timestamp`.

The generated processed file is `data/processed/windowed_labeled_data.csv`. It contains 252,720 rows and 78 columns and is about 168 MB. This processed CSV is a generated local artifact and should not be committed or included in the final ZIP.

The confirmed processed class balance is:

Class	Meaning	Windows	Percentage
0	Normal operation	247,520	97.94%
1	Failure-risk/anomaly	5,200	2.06%

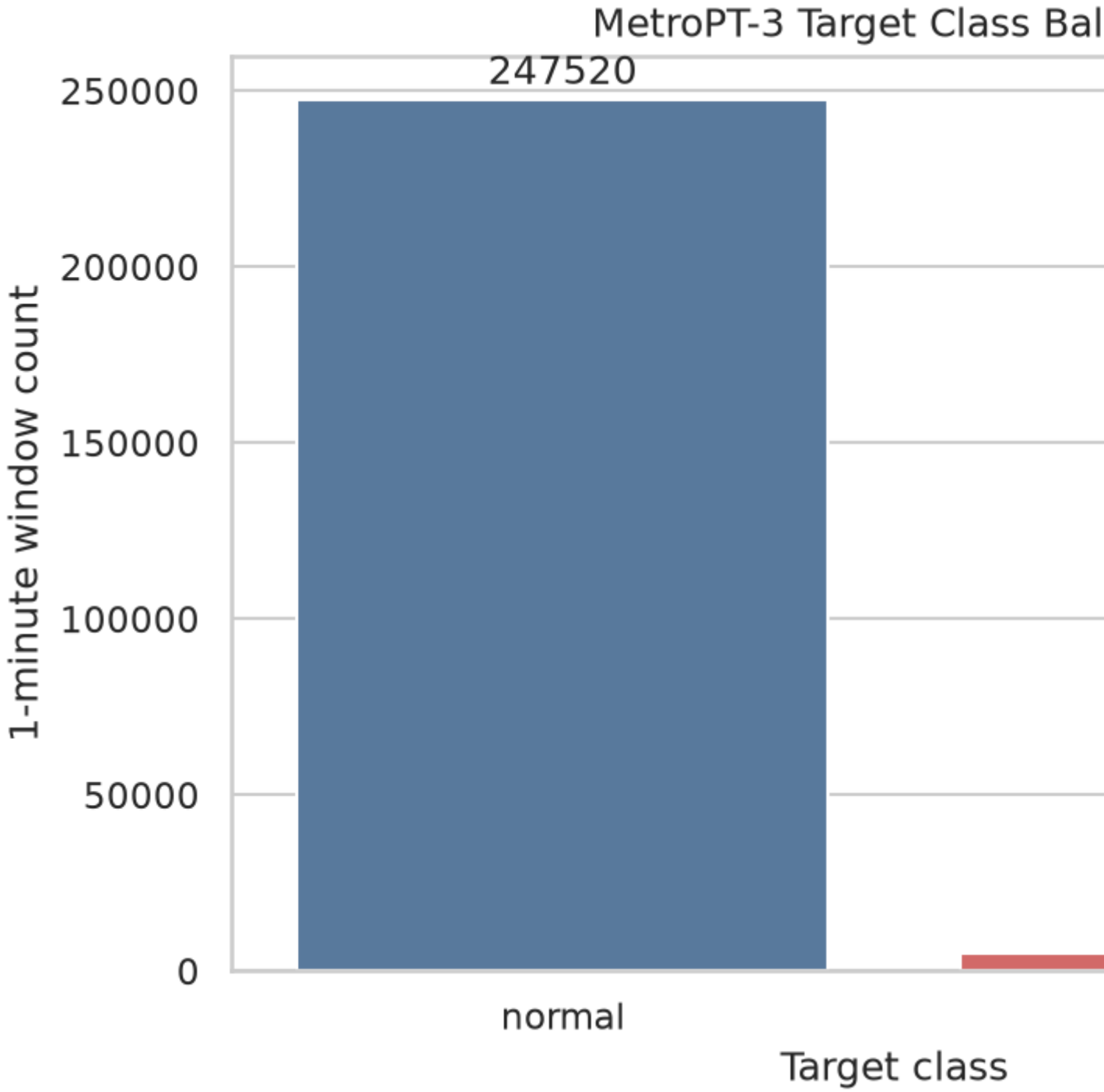


Figure 1. Processed class balance from `results/plots/class_balance.png`. The imbalance explains why accuracy is secondary.

The positive windows are concentrated in only 4 independent documented failure events:

Event	Split	Risk start	Failure start	Failure end	Positive windows
1	Train	2020-04-17 23:00:00	2020-04-18 00:00:00	2020-04-18 23:59:00	1,496
2	Train	2020-05-29 22:30:00	2020-05-29 23:30:00	2020-05-30 06:00:00	451
3	Validation	2020-06-05 09:00:00	2020-06-05 10:00:00	2020-06-07 14:30:00	2,923

Event	Split	Risk start	Failure start	Failure end	Positive windows
4	Test	2020-07-15 13:30:00	2020-07-15 14:30:00	2020-07-15 19:00:00	330

This table is saved in `results/event_summary.csv`. The 5,200 positive windows should not be interpreted as 5,200 independent failures. They are repeated 1-minute windows around the 4 documented events.

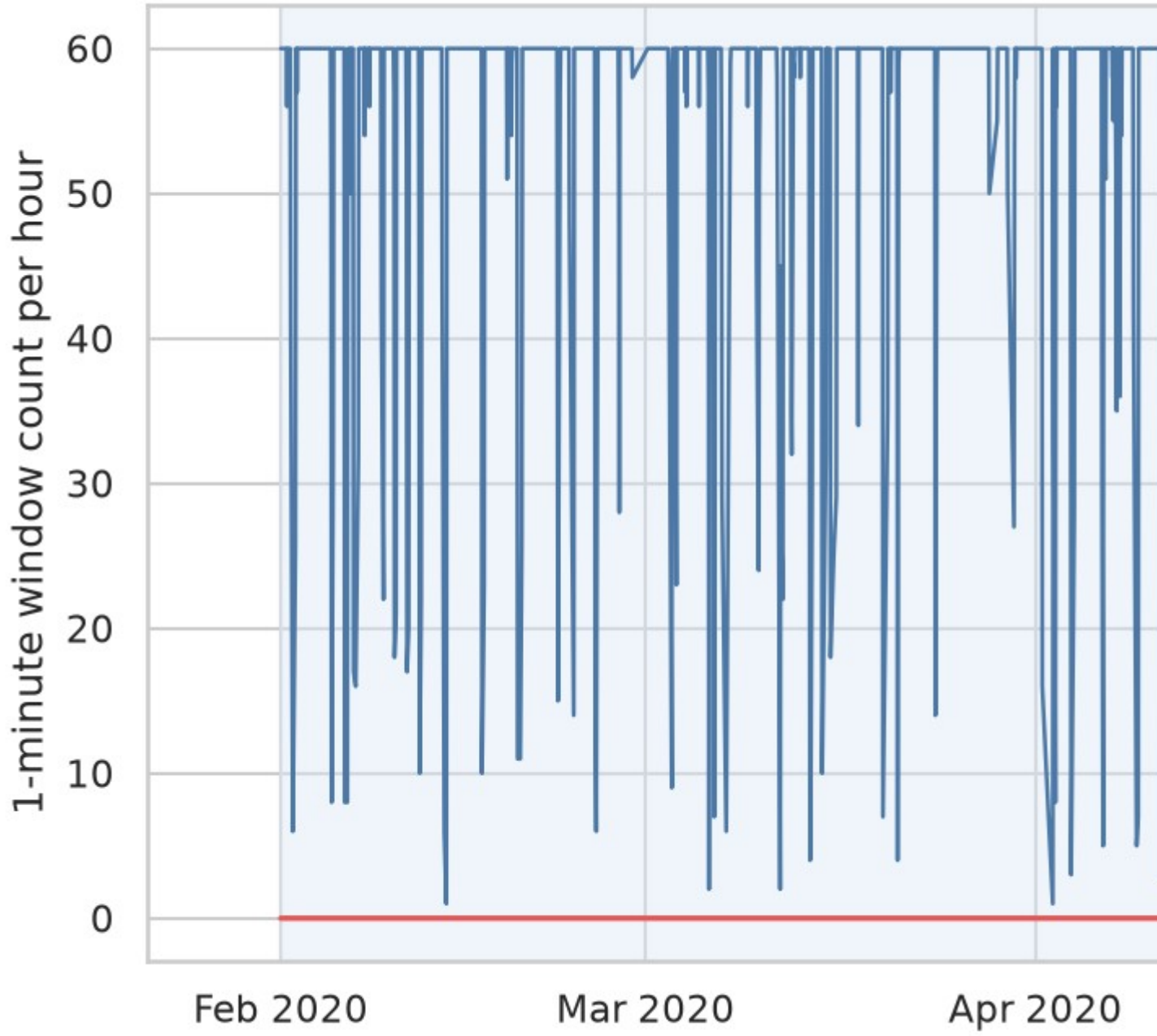


Figure 2. Failure-risk windows over time from `results/plots/failure_windows_timeline.png`. The four separated event clusters motivate the event-aware split.

5. Preprocessing and Exploratory Data Analysis

The preprocessing workflow loads the local MetroPT-3 CSV file, standardizes column names, detects and parses the timestamp column, converts raw timestamped sensor readings into 1-minute windows, and labels each window using the configured failure-risk periods.

For each numeric sensor column, the pipeline computes:

- Mean
- Standard deviation
- Minimum
- Maximum
- Last value

The pipeline also adds `row_count` for each 1-minute window. The final modeling table contains 76 numeric model features after excluding `window_start` and `target`. `window_start` is used only for ordering and split assignment. `target` is never used as an input feature.

Exploratory data analysis was generated with `src/eda.py`. The script produced the following report artifacts:

- `results/plots/class_balance.png`
- `results/plots/split_class_balance.png`
- `results/plots/sensor_correlation_heatmap.png`
- `results/plots/failure_windows_timeline.png`
- `results/plots/key_sensor_distributions.png`
- `results/eda_summary.csv`
- `results/event_summary.csv`

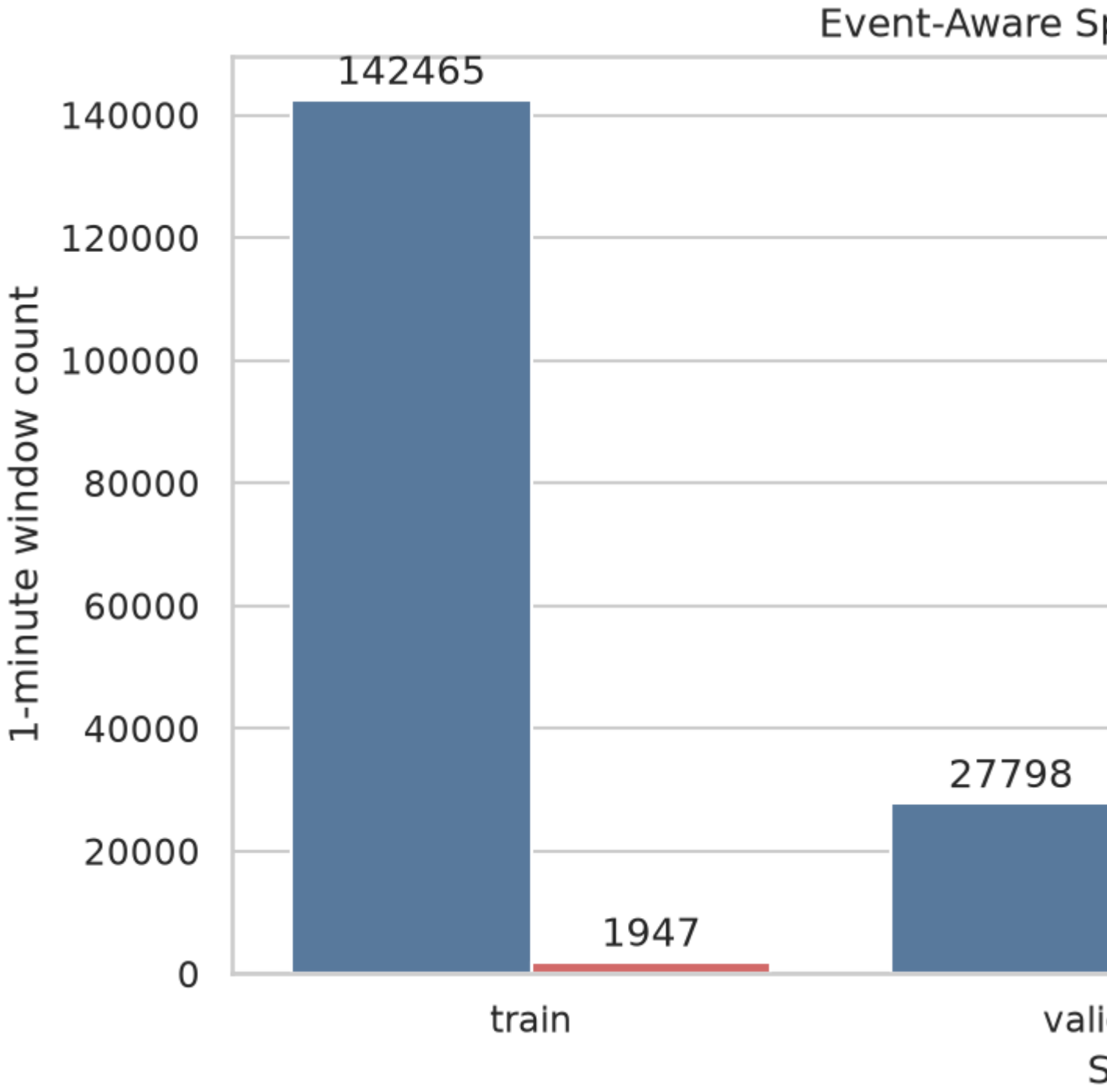


Figure 3. Train/validation/test class balance from `results/plots/split_class_balance.png`. The test split contains event 4 and only 330 positive windows.

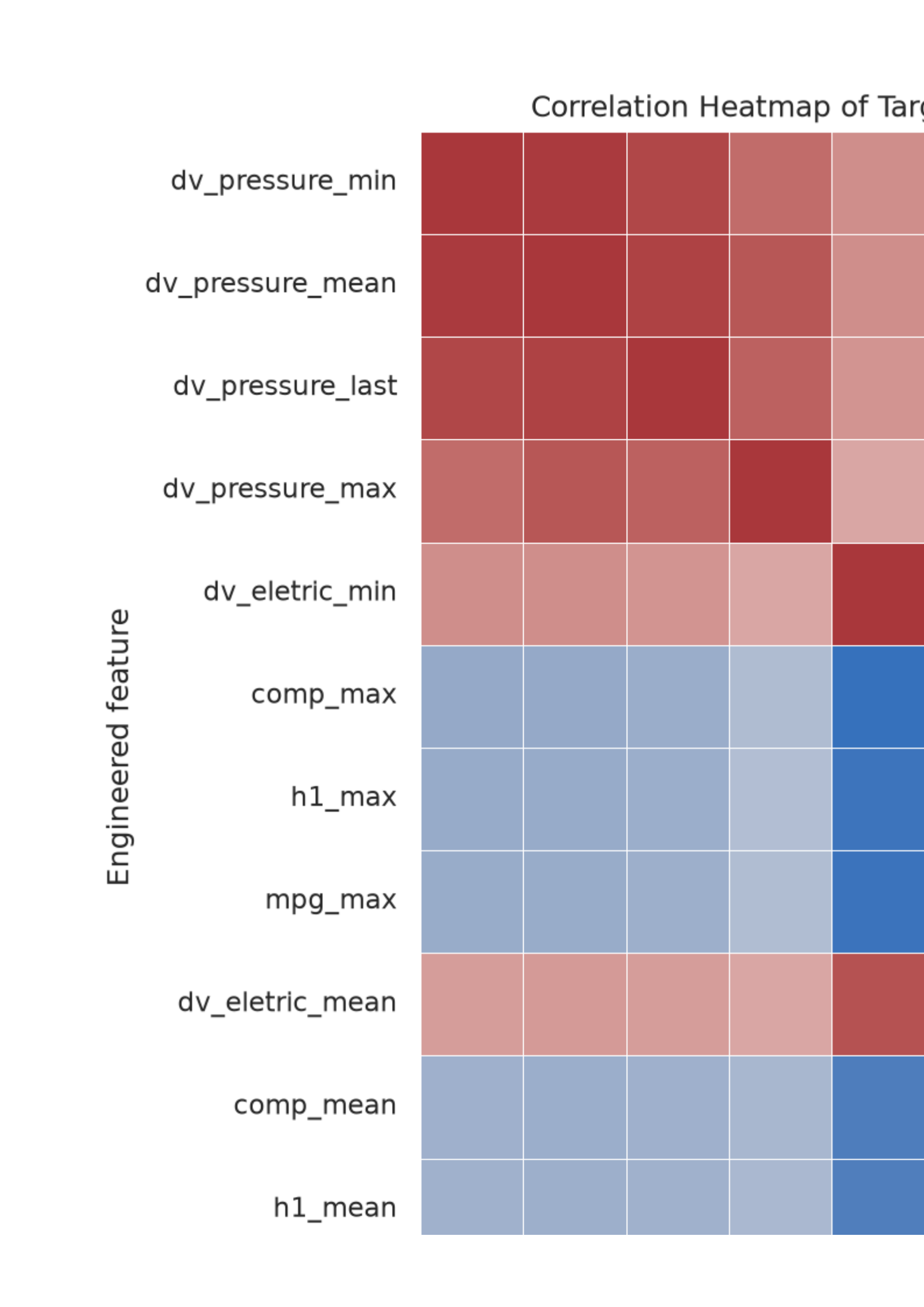
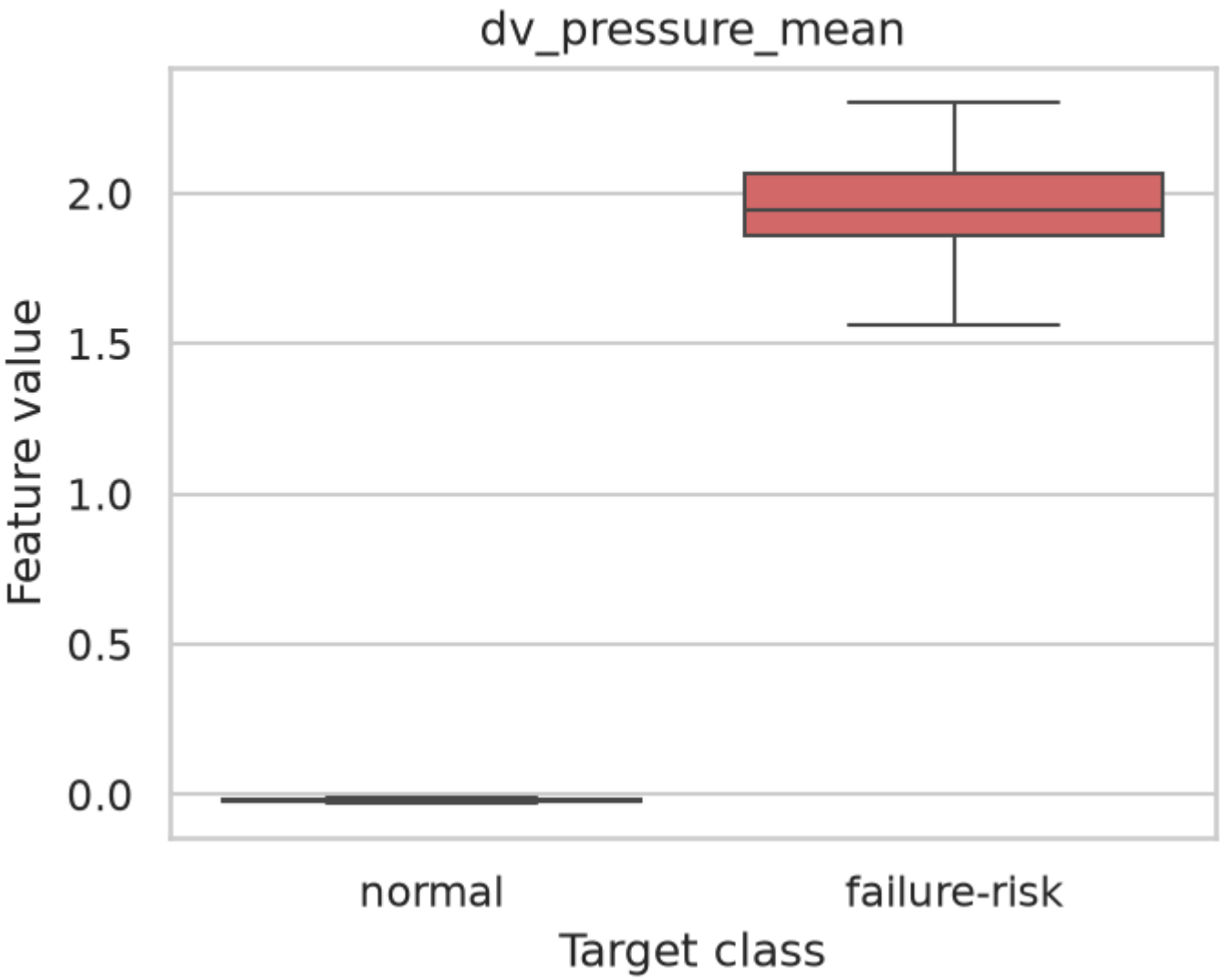

Correlation Heatmap of Tar
dv_pressure_min
dv_pressure_mean
dv_pressure_last
dv_pressure_max
dv_eletric_min
comp_max
h1_max
mpg_max
dv_eletric_mean
comp_mean
h1_mean
Engineered feature

Figure 4. Sensor correlation heatmap from `results/plots/sensor_correlation_heatmap.png`, focused on selected engineered features.


dv_pressure_mean
Feature value
2.0
1.5
1.0
0.5
0.0
normal
failure-risk
Target class

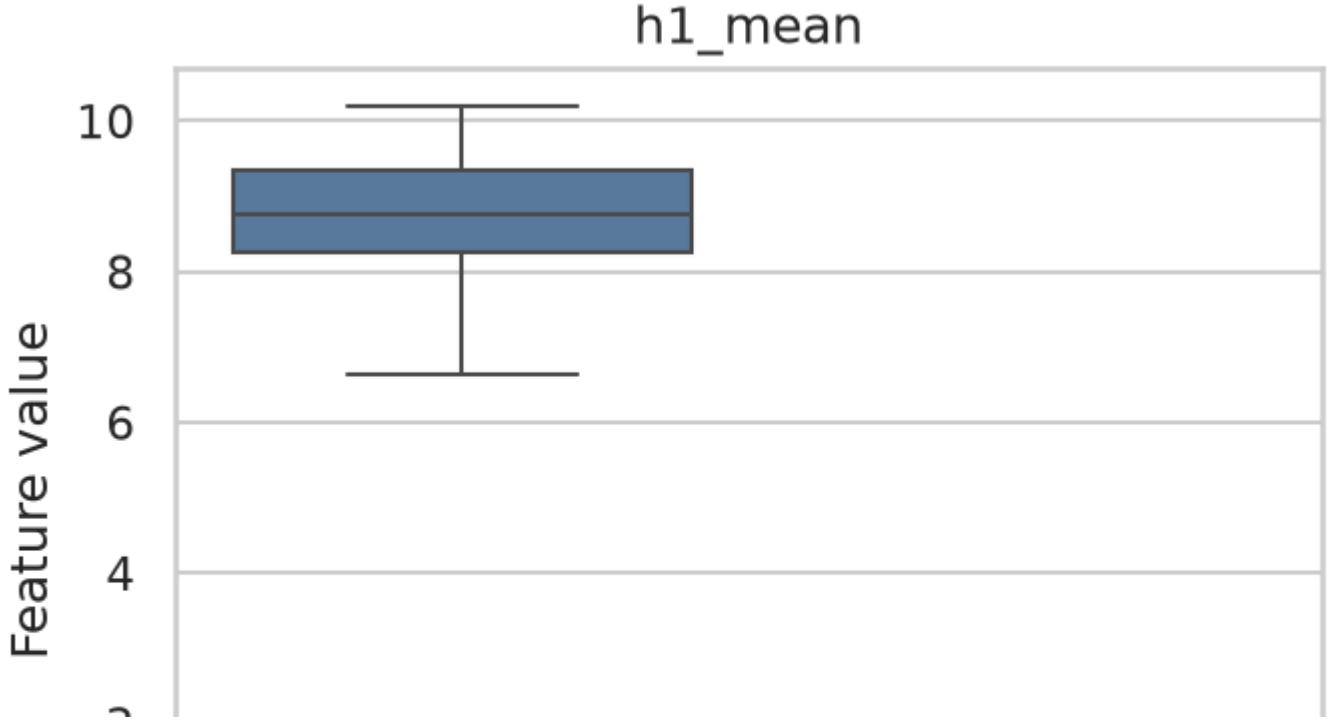

h1_mean
Feature value
10
8
6
4

Figure 5. Class-wise distributions for selected engineered sensor features from `results/plots/key_sensor_distributions.png`.

The EDA confirms severe class imbalance: 97.94% normal windows and 2.06% failure-risk windows. The failure-window timeline shows four separated clusters of positive windows, supporting the event-aware split instead of a random window-level split. The class-balance and split-balance plots also explain why accuracy is not the main metric.

Missing-value analysis from `results/eda_summary.csv` found 1,470 total missing cells across 15 standard-deviation feature columns. Each of these standard-deviation columns had 98 missing values: `tp2_std`, `tp3_std`, `h1_std`, `dv_pressure_std`, `reservoirs_std`, `oil_temperature_std`, `motor_current_std`, `comp_std`, `dv_eletric_std`, `towers_std`, `mpg_std`, `lps_std`, `pressure_switch_std`, `oil_level_std`, and `caudal_impulses_std`. These missing values are expected for windows where a standard deviation cannot be computed reliably, such as sparse windows. Model pipelines use median imputation for missing numeric values.

Outlier handling was limited. The existing EDA generated class-wise distribution plots for selected engineered sensor features, but it did not export numerical outlier counts and no outlier-removal rule was applied. This is intentional for the current report because extreme sensor readings may be meaningful failure-related behavior. The report therefore does not claim a verified number of outliers. Future work should quantify outliers with a documented rule and test whether retaining, winsorizing, or removing them improves event-aware recall without using the test event for tuning.

6. Methodology and Models

The project compares five supervised classifiers:

Model	Purpose
Logistic Regression baseline	Simple interpretable baseline with class weighting and scaling.
Decision Tree	Interpretable nonlinear model with tuned depth and leaf size.
Random Forest	Ensemble model for nonlinear sensor interactions.
Extra Trees	Ensemble baseline with randomized tree splits.
HistGradientBoostingClassifier	Boosted tree model for nonlinear relationships.

Class weighting is used where supported because the positive failure-risk class is rare. Logistic Regression uses a median imputer, standard scaler, and balanced class weights. Decision Tree, Random Forest, and Extra Trees use median imputation and balanced class weights. HistGradientBoostingClassifier is included as a boosted-tree model and was tuned using validation data only.

Feature interpretation is limited to outputs already generated by the project. The EDA selected the following engineered features as most target-correlated for the correlation heatmap: `dv_pressure_min`, `dv_pressure_mean`, `dv_pressure_last`, `dv_pressure_max`, `dv_eletric_min`, `comp_max`, `h1_max`, `mpg_max`, `dv_eletric_mean`, `comp_mean`, `h1_mean`, and `mpg_mean`. The class-wise distribution plot focuses on `dv_pressure_mean`, `dv_eletric_mean`, `comp_mean`, `h1_mean`, `mpg_mean`, and `dv_pressure_min`. These features suggest that engineered pressure and operating-state/electrical sensor

summaries are important descriptive signals in the processed data.

No reliable final feature-importance table was exported for the selected Logistic Regression model, and no saved tree-importance table is available in the current results. Because the final model failed to generalize to event 4, the report does not present unverified coefficient rankings as a reliable explanation of model behavior. The feature discussion should be read as descriptive EDA evidence, not as a causal or deployment-ready interpretation.

7. Validation and Hyperparameter Tuning Strategy

The main evaluation uses an event-aware blocked split:

- Training: failure events 1 and 2 plus surrounding normal windows
- Validation: failure event 3 plus surrounding normal windows
- Test: failure event 4 plus surrounding normal windows

The verified split sizes are:

Split	Windows	Class 0	Class 1	Failure events
Train	144,412	142,465	1,947	1 and 2
Validation	30,721	27,798	2,923	3
Test	77,587	77,257	330	4

This split is more realistic than random row-level splitting because it keeps each failure event entirely within one split. The held-out test set asks whether patterns learned from earlier failure events generalize to a later unseen event.

Decision Tree, Random Forest, and HistGradientBoostingClassifier were tuned with small hyperparameter grids. The best hyperparameters saved in the project review are:

Model	Best hyperparameters
Decision Tree	`classifier__max_depth=4`, `classifier__min_samples_leaf=1`
Random Forest	`classifier__max_depth=8`, `classifier__min_samples_leaf=1`
HistGradientBoostingClassifier	`classifier__learning_rate=0.05`, `classifier__max_leaf_nodes=31`

Decision thresholds were tuned on validation probabilities only, using thresholds from 0.01 through 0.99. The held-out event-4 test data was not used for model selection, hyperparameter selection, or threshold selection.

A secondary stratified window-level baseline is included only as an optimistic comparison. It uses a stratified random split over processed windows, so it can mix windows from the same failure events across train and test. It is useful as a feature-separability check, but it is not deployment-level performance.

8. Experimental Setup

The main scripts are:

Component	File	Command
Preprocessing	`src/preprocessing.py`	`python -m src.preprocessing`
Training and validation	`src/train.py`	`python -m src.train`
Final evaluation	`src/evaluate.py`	`python -m src.evaluate`
EDA	`src/eda.py`	`python -m src.eda`
Stratified baseline	`src/stratified_baseline.py`	`python -m src.stratified_baseline`
Demo	`demo/demo.py`	`python demo/demo.py`

Dependencies are listed in `requirements.txt`: pandas, NumPy, scikit-learn, matplotlib, seaborn, joblib, and Jupyter. The random state used in the project code is 42 where applicable.

The primary metrics are recall, F2-score, F1-score, precision, and the confusion matrix for the failure-risk class. Accuracy and ROC-AUC are also reported, but accuracy is not treated as the main metric because the data is highly imbalanced. False negatives are costly in train compressor maintenance, so recall and F2-score are more important than accuracy for interpreting the result.

The result tables used in this report are:

- `results/metrics_table.csv`
- `results/threshold_table.csv`
- `results/test_metrics.csv`
- `results/stratified_baseline_metrics.csv`
- `results/eda_summary.csv`
- `results/event_summary.csv`

9. Results and Model Comparison

The selected final model is the Logistic Regression baseline with threshold 0.61, chosen using validation-tuned F1-score. The same threshold also produced the best validation F2-score for this model.

Verified validation threshold comparison from `results/threshold_table.csv`:

Model	Default F1 at 0.50	Best F1 threshold	Best validation F1	Best F2 threshold	Best validation F2
Logistic Regression baseline	0.7550	0.61	0.8216	0.61	0.8945
Decision Tree	0.5001	0.13	0.5001	0.13	0.3856
Random Forest	0.5313	0.31	0.5525	0.19	0.4790
Extra Trees	0.0409	0.04	0.6952	0.03	0.7717
HistGradientBoostingClassifier	0.6603	0.11	0.6652	0.11	0.5595

Verified final event-aware test metrics on held-out failure event 4 from `results/test_metrics.csv`:

Metric	Value
Accuracy	0.9942902805882429
Precision	0.0
Recall	0.0
F1-score	0.0
F2-score	0.0
ROC-AUC	0.12087550368094527

Final event-aware test confusion matrix counts for labels `[normal, failure_risk]`:

Actual / predicted	Normal	Failure-risk
Normal	77,144	113
Failure-risk	330	0

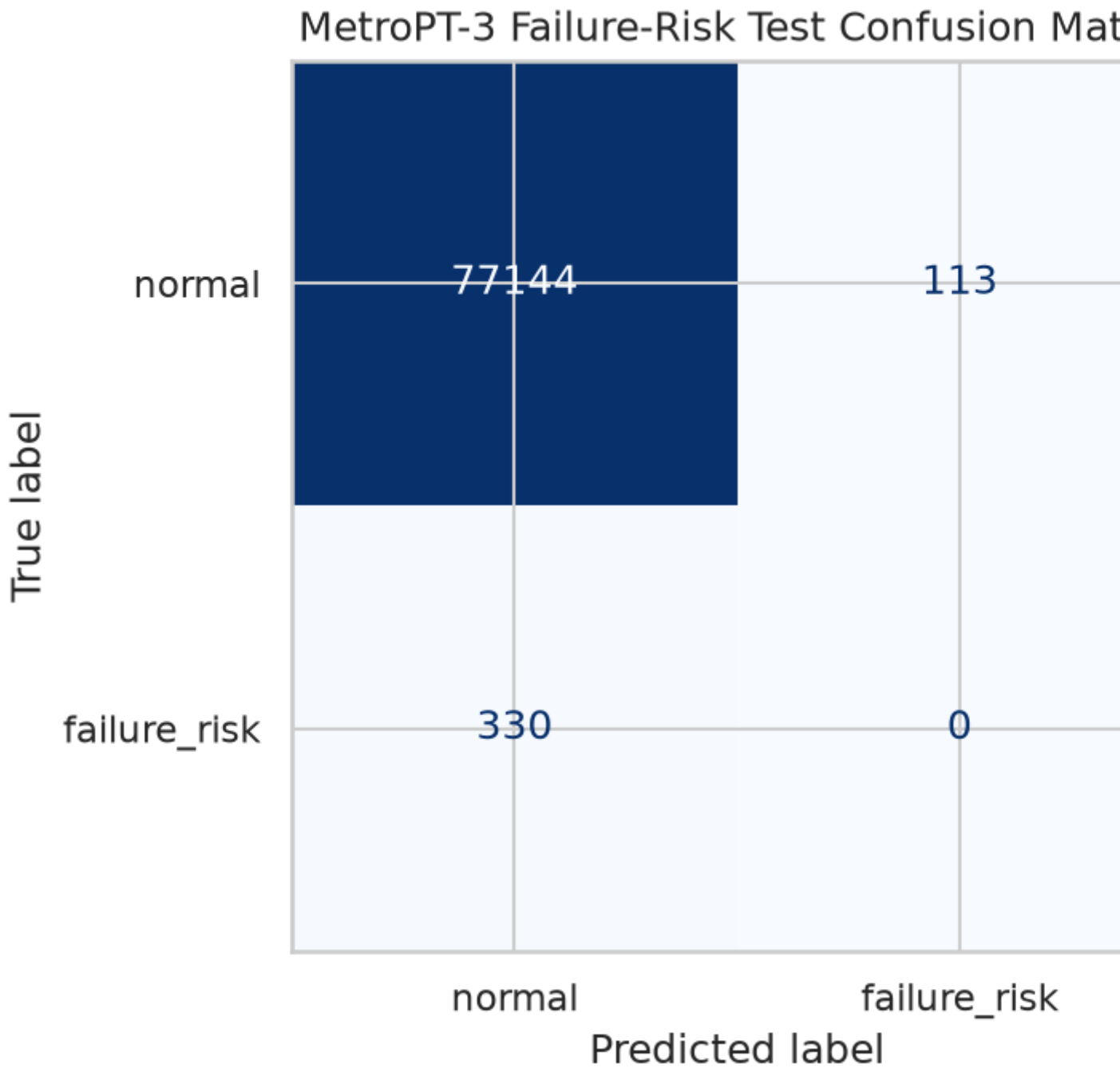


Figure 6. Final event-aware test confusion matrix from `results/plots/confusion_matrix.png`. Held-out event 4 has 330 false negatives and 0 true positives.

The high event-aware test accuracy is misleading. The held-out event-4 test block contains many more normal windows than failure-risk windows, and the selected model did not detect any of the 330 true failure-risk windows.

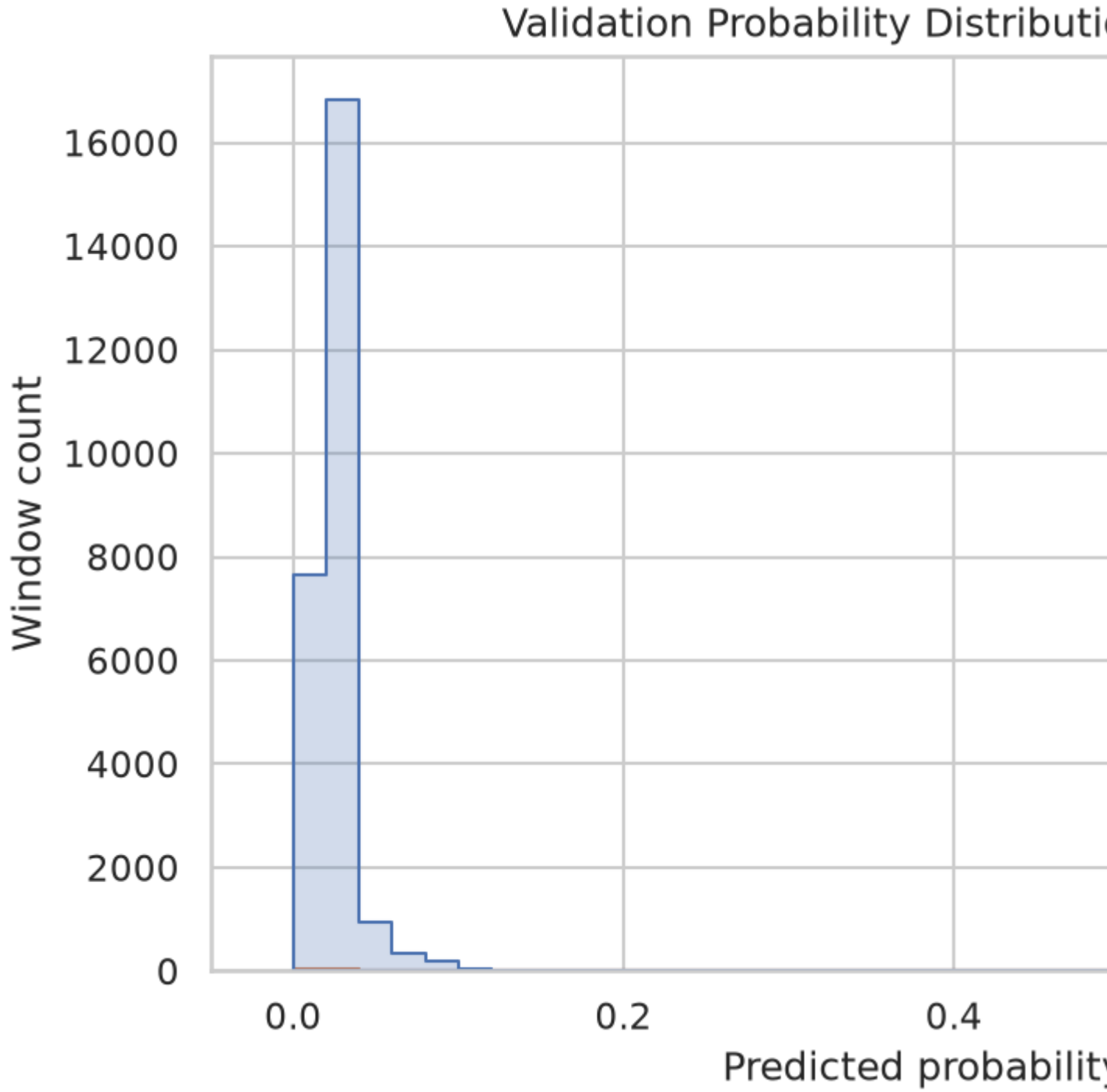


Figure 7. Validation probability distribution from `results/plots/validation_probability_distribution.png`.

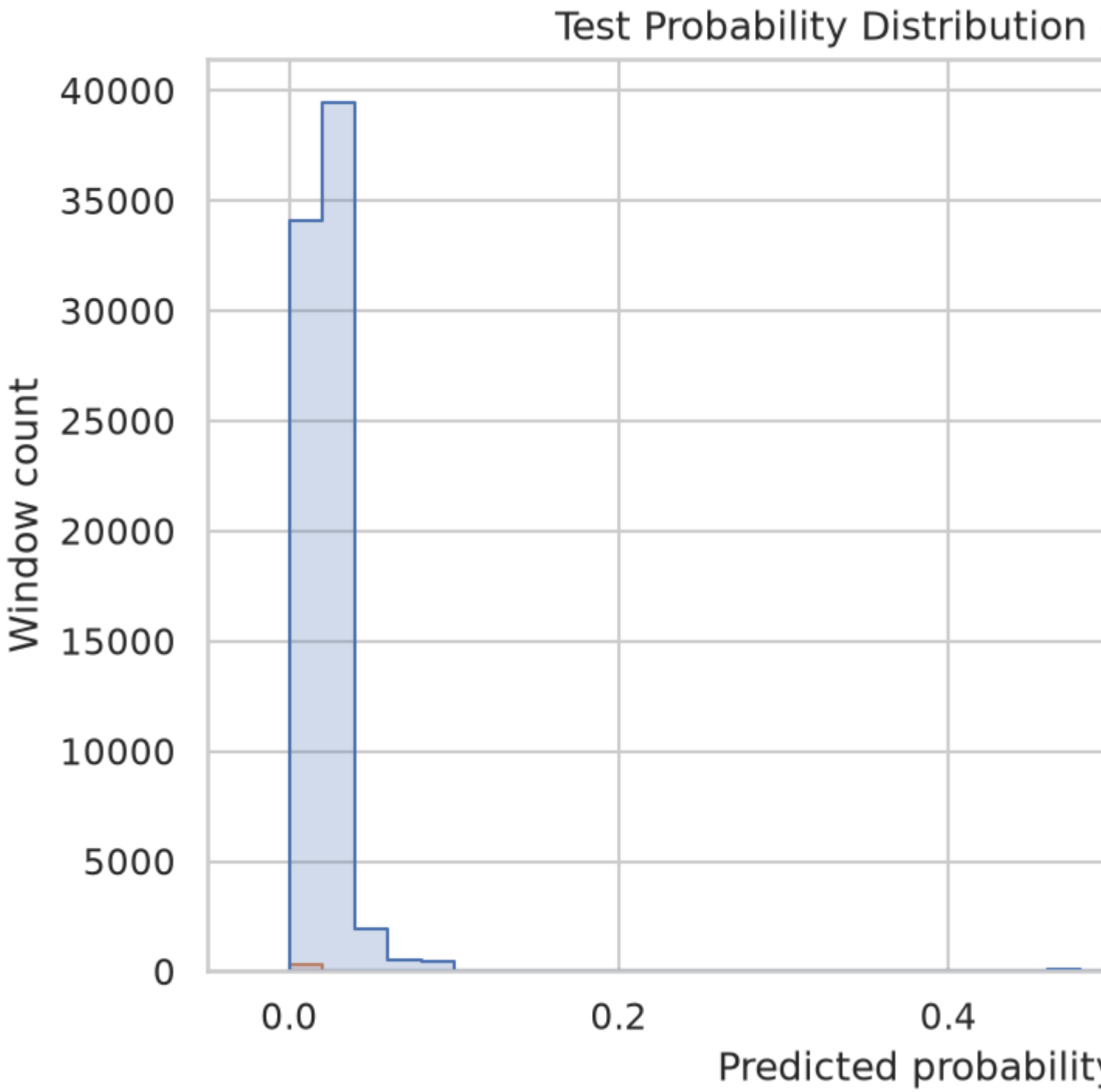


Figure 8. Test probability distribution from `results/plots/test_probability_distribution.png`.

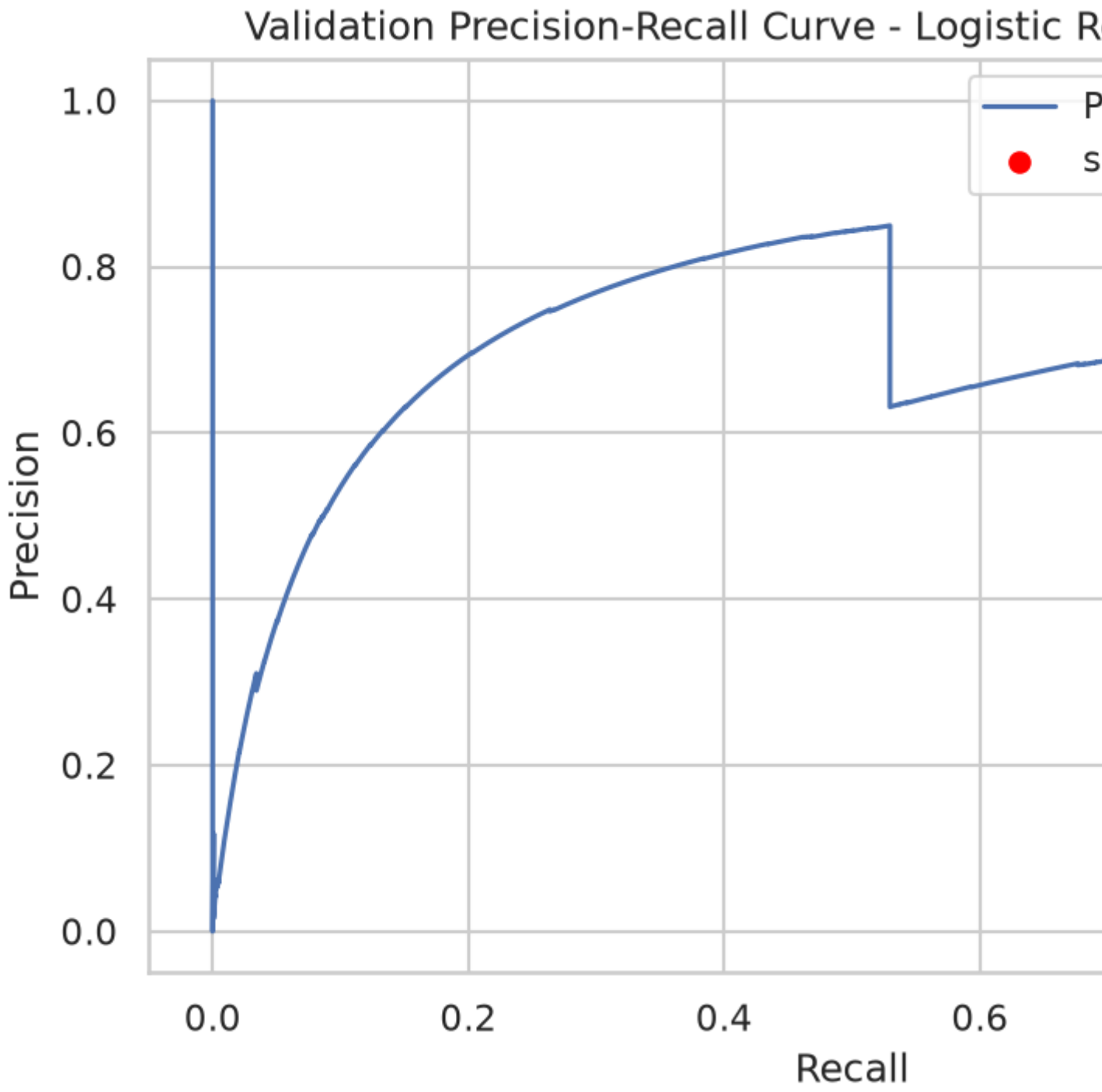


Figure 9. Validation precision-recall curve from `results/plots/precision_recall_curve.png`.

The secondary stratified window-level baseline produced much stronger results:

Model	Accuracy	Precision	Recall	F1	F2	ROC-AUC
Logistic Regression baseline	0.9636	0.3573	0.9615	0.5210	0.7185	0.9849
Decision Tree	0.9951	0.8421	0.9385	0.8877	0.9175	0.9687
Random Forest	0.9963	0.8638	0.9760	0.9165	0.9513	0.9950
Extra Trees	0.9963	0.8695	0.9673	0.9158	0.9460	0.9926

Model	Accuracy	Precision	Recall	F1	F2	ROC-AUC
HistGradientBoostingClassifier	0.9955	0.8301	0.9817	0.8996	0.9471	0.9986

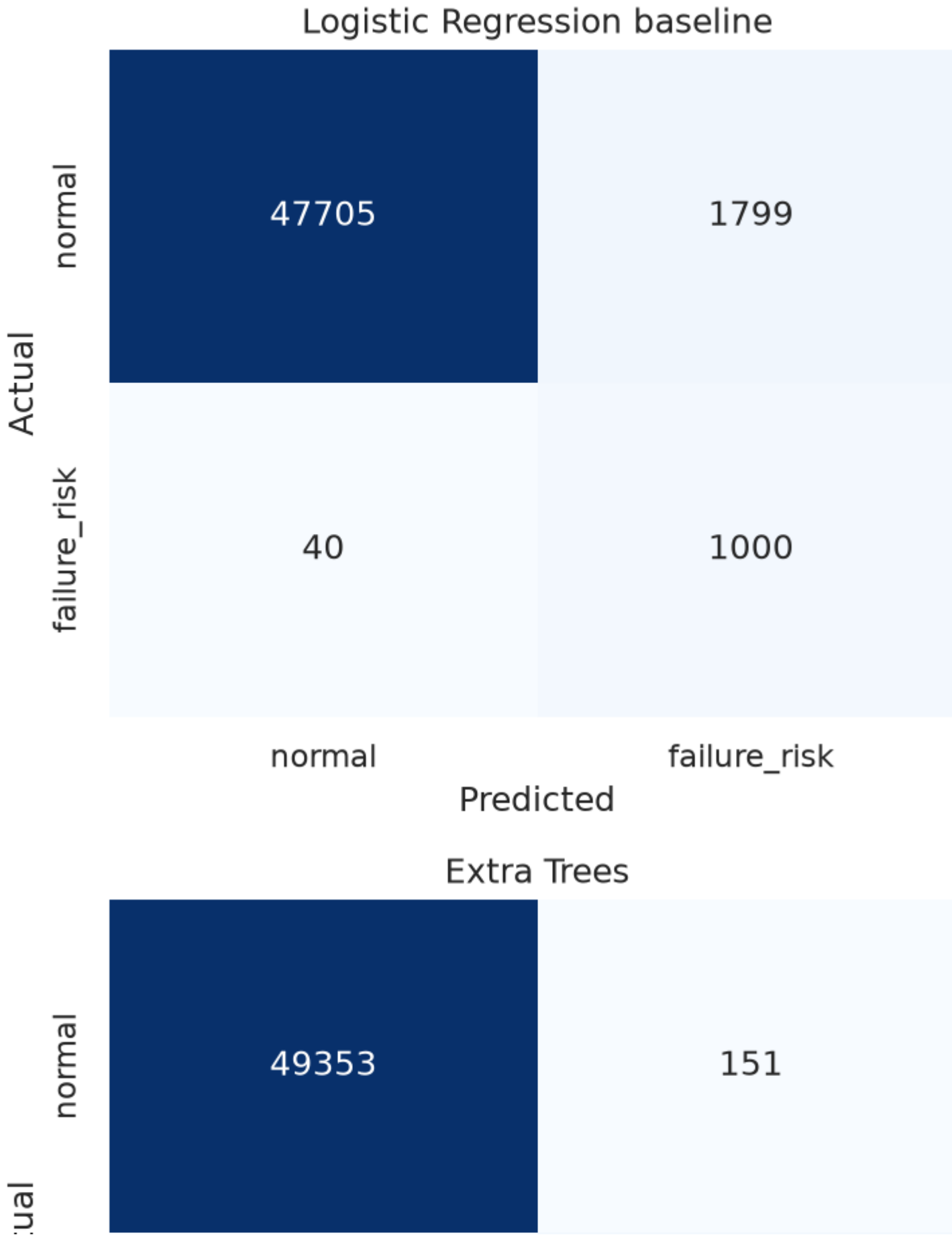

Logistic Regression baseline
Actual
normal
failure_risk
47705
1799
40
1000
normal
failure_risk
Predicted
Extra Trees
normal
49353
151

Figure 10. Stratified window-level baseline confusion matrix from `results/plots/stratified_baseline_confusion_matrix.png`. This baseline is optimistic only.

These stratified results are optimistic only. They come from a random window-level split and can mix windows from the same failure event across train and test. They show that the engineered features can separate many labeled windows when event independence is not enforced, but they do not replace the event-aware held-out result.

10. Error Analysis and Qualitative Discussion

The most important error is the complete miss on held-out event 4. At threshold 0.61, the selected Logistic Regression model predicted all 330 true event-4 failure-risk windows as normal. These are false negatives, and in predictive maintenance they are the most serious error type because they represent missed warning windows.

The likely explanation is poor generalization across independent failure events. The model learned from events 1 and 2 and selected its threshold on event 3, but event 4 did not produce a pattern that the selected model could recover at the validation-selected threshold. This interpretation is supported by the verified event-aware test metrics: recall, F2-score, F1-score, and precision are all 0.0 for the failure-risk class.

The model also produced 113 false positives on event 4's test block. These are normal windows predicted as failure-risk. In an operational setting, false positives could cause unnecessary inspections, operator attention, or maintenance planning cost. They are less severe than false negatives, but too many false positives could still reduce trust in the system.

The validation results looked much stronger than the final test result. On validation event 3, the selected model at threshold 0.61 produced 2,779 true positives and 144 false negatives. On held-out event 4, it produced 0 true positives and 330 false negatives. This gap shows that validation event 3 was not enough to guarantee generalization to a future failure event.

The current model is not deployment-ready. A deployment-ready failure-risk system would need evidence that it can reliably catch future independent failure events. This project shows the opposite for the final event-aware test: event 4 generalization failed.

11. Demo or Usage Demonstration Description

The command-line demo is implemented in `demo/demo.py` and is run from the repository root:

```
python demo/demo.py
```

The demo loads `data/processed/windowed_labeled_data.csv`, drops `window_start` and `target`, keeps the engineered 1-minute window features used by the training pipeline, loads `models/final_model.joblib`, reads the selected threshold from `results/threshold_table.csv`, and prints representative predictions for a normal window and a failure-risk window when both are available.

The saved demo output in `results/demo_output.txt` shows:

- Feature columns used: 76
- Threshold used: 0.61

- Normal sample at `2020-02-01 00:00:00`: predicted normal with probability 0.016487
- Failure-risk sample at `2020-04-17 23:00:00`: predicted failure-risk with probability 0.641494

This demo illustrates how to use the processed-feature prediction pipeline. It is not a deployment demo and should not be presented as production-ready because event-aware testing failed to generalize to held-out failure event 4.

12. Limitations and Future Work

The main limitation is the small number of independent failure events. Although the processed dataset contains 5,200 positive windows, those windows come from only 4 documented failures. This makes supervised generalization difficult and makes random window-level evaluation overly optimistic.

Other limitations include:

- The labels depend on documented failure start and end times.
- The one-hour lead window is a project-defined early-warning assumption.
- The final model failed to identify event 4 at the validation-selected threshold.
- No reliable final model feature-importance artifact was generated.
- Outlier analysis was qualitative and plot-based only; no numerical outlier counts were exported.
- Dataset-specific license or usage terms should be reviewed directly from the UCI source before public redistribution.

Future work should focus on improving held-out event recall without tuning on the test set. Useful directions include collecting more failure events, applying blocked cross-validation across more event groups, testing alternate window sizes, comparing supervised learning with anomaly-detection or semi-supervised approaches, exporting validated feature-importance artifacts, and quantifying outliers with a documented rule. Any future improvement must continue to keep the held-out test event separate from model and threshold selection.

13. Conclusion

This project provides a reproducible supervised learning workflow for MetroPT-3 compressor failure-risk prediction. It creates verified 1-minute labeled windows, compares five required models, tunes selected models and thresholds using validation data only, and reports metrics that reflect predictive-maintenance priorities.

The main realistic result is the event-aware held-out evaluation. That result shows failed generalization to failure event 4: the selected model had 0.0 precision, recall, F1-score, and F2-score for the failure-risk class, with 330 false negatives and 0 true positives. The high accuracy is a consequence of class imbalance and should not be interpreted as successful failure-risk prediction.

The stratified window-level baseline is useful only as an optimistic comparison. It demonstrates that random window-level splits can produce much stronger metrics when windows from the same failure events may be mixed across train and test. For this project, the event-aware evaluation remains the realistic result, and the current model is not deployment-ready.

14. Team Contribution Section

Exact team member names were not available during final report preparation. Replace the `TODO_NAME`

placeholders with verified names before submission.

- Member 1: TODO_NAME — project coordination, GitHub/repository organization, README, report integration, final submission packaging
- Member 2: TODO_NAME — dataset source documentation, MetroPT-3 failure window verification, labeling logic validation
- Member 3: TODO_NAME — preprocessing, 1-minute window feature engineering, EDA figures, class imbalance analysis
- Member 4: TODO_NAME — model training, hyperparameter tuning, event-aware validation, threshold tuning, model comparison tables
- Member 5: TODO_NAME — evaluation, error analysis, stratified baseline experiment, demo script, reproducibility review

All team members are responsible for understanding the submitted code, labels, split strategy, metrics, limitations, and final conclusions.

15. References

Academic Integrity and Tool Acknowledgment

The MetroPT-3 dataset and the Python libraries used in this project are external sources and are acknowledged below. AI-assisted tools, including ChatGPT/Codex, were used for coding support, debugging, organization, documentation review, and drafting assistance. The team is responsible for understanding all submitted code and results and for verifying that reported metrics and figures come from saved project outputs.

References

[1] UCI Machine Learning Repository, "MetroPT-3 Dataset." https://archive.ics.uci.edu/dataset/791/metropt+3+dataset

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011. https://scikit-learn.org/

[3] The pandas development team, "pandas." https://pandas.pydata.org/

[4] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020. https://numpy.org/

[5] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007. https://matplotlib.org/

[6] M. L. Waskom, "seaborn: statistical data visualization," Journal of Open Source Software, vol. 6, no. 60, 3021, 2021. https://seaborn.pydata.org/

[7] COEN 330 Applied Machine Learning project guidelines, course-provided PDF: `docs/COEN330_Project_Guidelines.pdf`.